Viewstamped Replication Made Live

Pekka Enberg

Draft of September 6, 2026

Abstract

Viewstamped Replication is a protocol for replicating a state machine across a cluster of machines that can crash: a primary orders the operations, a quorum of backups acknowledges them, and a view change replaces the primary when it fails. It was introduced by Oki and Liskov in 1988 and revised by Liskov and Cowling in 2012, which adds a recovery protocol that needs no disk. The 2012 presentation has known bugs: its recovery can lose committed data, its state transfer can lose committed data, and its commit-number handling can execute an operation twice. This paper formalises the protocol with the errata applied, as a transition system and as pseudocode, and proves it safe: in every reachable state, two replicas that have both committed an op-number hold the same entry there, and every committed entry is held by enough replicas to be in every quorum they could form. The proof is an inductive invariant over the history of every message ever sent, and it is machine-checked in Lean 4.

Keywords: Viewstamped Replication, state machine replication, consensus, inductive invariants, Lean

1 Introduction

Viewstamped Replication keeps a service available by running the same deterministic operations, in the same order, on several machines. One replica, the primary, assigns each client operation a position in a log and sends it to the others; once a quorum has acknowledged it, the operation is committed and every replica executes it. If the primary fails, the backups move to a new *view* with a new primary chosen by a fixed rule, and the new primary starts from a log that a quorum has seen, so nothing committed is lost. Oki and Liskov introduced the protocol in 1988 [3], inside a transactional system. Liskov and Cowling revised it in 2012 [1], presenting it on its own, with a state transfer protocol for replicas that fall behind and a recovery protocol that lets a replica rejoin after losing all its memory without ever having written to disk.

The 2012 presentation has known bugs. Its diskless recovery lets a replica forget it took part in a view change and complete that change later from stale messages, which loses committed data [2]. Its state transfer truncates a log below an entry the replica has already acknowledged, which also loses committed data, and its commit-number handling lets a replica adopt a lower commit number and execute an operation twice [4]. Each has a known fix. An implementation must apply all of them, and then runs a protocol that no single paper states.

This paper states that protocol and proves it safe. Section 2 formalises the cluster as a transition system; Sections 4 to 6 give the handlers as pseudocode with every erratum applied and marked; Appendix A walks through the published protocol and explains each correction where the paper's text leads wrong. The proof is an inductive invariant over the history of every message ever sent, and it is machine-checked in Lean 4 (Appendix B).

1.1 Our result

Fix a cluster of $N \geq 2$ replicas running the protocol of Sections 4 to 6 on an asynchronous network that may lose, delay, duplicate, and reorder messages, in which replicas may crash and

restart with nothing but their view number. Let k_r be the commit number and L_r the log of replica r, and let $\mathcal{R}$ be the set of replicas not currently recovering (Section 2).

Theorem 1.1 (Log safety)**.** *In every reachable cluster state:*

(i) Prefix agreement. *If $n \le k_a$ and $n \le k_b$ then $L_a[n] = L_b[n]$.*

(ii) Durability. *For every $r \in \mathcal{R}$ and $n \le k_r$, at least $|\mathcal{R}| + 1 - Q$ replicas in $\mathcal{R}$ hold $L_r[n]$ at op-number n.*

(iii) Commit bounded. $k_r \le |L_r|$.

Durability says that every quorum the non-recovering replicas could form contains a replica holding the committed entry; with nobody recovering the bound is a majority. A recovering replica holds nothing and votes for nothing, so it counts on neither side.

The proof in the body of this paper is a hand proof at the level of detail of an algorithms paper. Every definition, lemma, and theorem in it has a counterpart in a Lean 4 development that proves Theorem 1.1 in full from the same invariant, with no unproved step; Appendix B states the mechanised theorem, its assumptions, and the correspondence.

1.2 Technique

The theorem is not preserved by a single protocol step on its own. When an acknowledgement completes a quorum and an entry becomes committed, whether it is now held by a majority depends on what the other acknowledgements said, and nothing in the theorem records that. We strengthen the statement into an inductive invariant $\mathcal{I}$ (Section 2.3) that holds initially, is preserved by every step, and implies the theorem.

$\mathcal{I}$ is stated over the *history*: the set of every message ever sent, which delivery never removes from. Replicas forget, since a view change replaces a backup's log, but the acknowledgement the backup sent stays in the history, and the invariant quantifies over that. The history also makes the network model one rule: delivery picks any message ever sent, which covers loss, delay, duplication, and reordering at once.

$\mathcal{I}$ has five parts, named for what they say. Replica-consistency: each replica's own fields are consistent with one another. Message-consistency: each message's fields agree with one another. View-agreement: within a view, an op-number determines its entry, so every fragment of a view's log that the history holds agrees with every other. Commit-quorums: every commit number, on a replica or in a message, has a quorum of acknowledgements behind every op-number up to it. Commit-survival: whatever was committed in a view is held, at the same op-number, by every later view. Around these are the clauses the induction turned out to need, most of them about the order in which things happened: a vote is cast after the acknowledgements its sender made, a view is started at most once, a primary's log in its view never shrinks. The lemma that carries the induction is the following.

Lemma 1.2 (The chosen log holds earlier commits)**.** *Suppose entry e was acknowledged at op-number n in view v by a quorum, and view $u > v$ was started from a quorum of votes V, and every vote in V from a view strictly between v and u holds e at n. Then the vote of V from which u's log was chosen holds e at n.*

The two quorums intersect in a replica x. The vote x cast for u was cast after x acknowledged n in v, so it is from a view at least v; if from v itself, it covers the acknowledgement; if from a later view, the hypothesis applies. The best vote is at least as good as x's, by the ordering the new primary uses, and holds e too. A strong induction on u discharges the hypothesis. Each of the three safety corrections in Appendix A is what makes one step of this argument go through: without a persisted view number a replica's view can go backwards; without the ban on truncation a vote need not cover what its sender acknowledged; without monotone commits Commit-quorums fails at the replica that adopted a lower number.

1.3 Organisation

Section 2 fixes notation, the model of a cluster, and the invariant. Section 3 states the interface of every handler as a lemma and proves the theorem from those interfaces, before any handler is shown. Sections 4 to 6 give the handlers as pseudocode, one section per sub-protocol, each with an overview and the analysis that proves its interface lemmas. Appendix A is the published protocol and its corrections. Appendix B is the mechanised proof.

2 Preliminaries

2.1 The cluster

Definition 2.1 (Configuration)**.** $N \geq 2$ replicas with identities $0, \ldots, N-1$; $f = \lfloor N/2 \rfloor$; the quorum size $Q = f + 1$; and primary$(v) = v \bmod N$, the primary of view v.

Definition 2.2 (Log)**.** A log L is a finite sequence of entries $e = (c, s, op)$, client identity, request number, operation. The position of an entry in the log is its *op-number*, counted from 1: $L[n]$ is the entry with op-number n, for $1 \leq n \leq |L|$. $L(a..b]$ is the entries with op-numbers $a+1, \ldots, b$, and $L \cdot e$ is the log with e appended, at op-number $|L| + 1$.

Definition 2.3 (Replica)**.** A replica r has an identity $\mathrm{id}_r < N$; a status $status_r$, one of NORMAL, VIEW-CHANGE, TRANSFER, or RECOVERING; a view number v_r; a last normal view ℓ_r, the latest view in which $status_r$ was NORMAL; a log L_r with $n_r = |L_r|$; a commit number k_r; a flag $catching_r$; a client table C_r mapping client identities to a request number and an optional reply; acknowledgements A_r mapping op-numbers to sets of identities; a set S_r of identities; a flag $voted_r$; votes V_r mapping identities to votes; a nonce x_r; recovery responses R_r mapping identities to responses; and a state machine state σ_r. isPrimary(r) means $\mathrm{id}_r = \mathrm{primary}(v_r)$.

Definition 2.4 (Messages)**.** Eleven kinds. i is an identity, v a view, n an op-number, k a commit number, L a log, x a nonce, and $st \in \{\bot\} \cup \{(L, k)\}$ an optional state.

Request(c, s, op)	StartViewChange(v, i)
Prepare(v, n, e, k)	DoViewChange(v, i, ℓ, L, n, k)
PrepareOk(v, n, i)	StartView(v, L, n, k)
Commit(v, k)	Recovery(i, x, v)
GetState(i, v, n)	RecoveryResponse(v, x, i, st)
NewState(v, L, a, b, k)	

A *vote* is the triple (ℓ, L, k) a DoViewChange carries: the sender's last normal view, its log, its commit number.

Definition 2.5 (Cluster)**.** A cluster state is $\Sigma = (replicas, sent, replies, started)$: the replicas by identity; *sent*, the set of every (to, m) ever sent; *replies*, every reply (v, c, s, x) ever produced; and *started*, the set of (v, V) for every view v some primary started and the votes V it chose the log from. Sent(to, m) means $(to, m) \in sent$. $\mathcal{R}$ is the set of replicas with $status \neq$ RECOVERING.

Definition 2.6 (Step)**.** $\Sigma \to \Sigma'$ is one of: *deliver* some $(to, m) \in sent$ to replica to; *idle* at some replica; *request* (c, s, op) at some replica; *recover* some replica with the view number it holds in Σ and a nonce that no Recovery in *sent* carries. The replica runs the handler of Sections 4 to 6; everything it sends, replies, and starts is added to *sent*, *replies*, and *started*, which therefore only grow.

Definition 2.7 (Reachable)**.** The initial cluster has every replica with $status =$ NORMAL, $v = \ell = k = 0$, L empty, every map and set empty, σ the initial state, and the three sets of Σ empty. A cluster is *reachable* if it is obtained from the initial cluster by finitely many steps.

Assumption 2.8. *Throughout:*

(i) $N \geq 2$.

(ii) The environment chooses which replica takes an idle step and when; the protocol's timers count those idle steps. Every schedule is a reachable execution.

(iii) A replica's view number v_r is written to stable storage after every step and before anything the step produced is sent, and is the only state a recover *step retains.*

(iv) The nonce a recover *step uses is fresh: no* RECOVERY *in the history carries it.*

Remark 2.9. (i) excludes only the one-replica cluster, which never sends a message, so nothing about it can be stated through *sent*; it is trivially safe. (ii) means safety is proved for every timing, including ones no timer would produce. (iii) is the one erratum to the published protocol's diskless design, and Appendix A.5 shows it cannot be dropped. (iv) is the paper's "nonce" (§4.3), which a clock or a persisted counter provides.

Lemma 2.10 (Quorum intersection)**.** *Two sets of at least Q identities below N have a common element. More precisely, two sets X, Y of identities below N share at least $|X|+|Y|-N$ elements.*

Proof. $|X \cap Y| = |X| + |Y| - |X \cup Y| \geq |X| + |Y| - N$, and $2Q = 2\lfloor N/2 \rfloor + 2 > N$. □

Delivery chooses a message from *sent* and leaves it there, so a message may arrive any number of times, in any order, or never. A replica that is never chosen to run is a crashed one.

2.2 The history

The invariant is stated over the history. The following definitions name the pieces of a view's log that the history holds.

Definition 2.11 (Vote of)**.** Vote(u, x, d): replica x cast the vote $d = (\ell, L, k)$ for view u. Either the history holds DOVIEWCHANGE$(u, x, \ell, L, |L|, k)$, or $(u, V) \in$ *started* with $(x, d) \in V$. The second case covers the new primary's own vote, which it records without sending.

Definition 2.12 (Fragment)**.** Frag(v, a, L): the history holds L as a piece of view v's log, the entries with op-numbers $a+1, \ldots, a+|L|$. One rule per kind of message that carries log:

$$\begin{array}{ll}
\text{Sent}(_, \textsc{Prepare}(v, n, e, _)) & \Rightarrow \text{Frag}(v, n-1, [e]) \\
\text{Sent}(_, \textsc{NewState}(v, L, a, _, _)) & \Rightarrow \text{Frag}(v, a, L) \\
\text{Sent}(_, \textsc{StartView}(v, L, _, _)) & \Rightarrow \text{Frag}(v, 0, L) \\
\text{Sent}(_, \textsc{RecoveryResponse}(v, _, _, (L, _))) & \Rightarrow \text{Frag}(v, 0, L) \\
\text{Vote}(_, _, (\ell, L, _)) & \Rightarrow \text{Frag}(\ell, 0, L)
\end{array}$$

Remark 2.13. A vote is a fragment of the view its sender was *last normal in*, ℓ, not of the view it votes for. A replica catching up with a view has a view number ahead of its log; tagging by ℓ is what keeps it consistent with the rest.

Definition 2.14 (Holds)**.** Holds(v, n, e): there are $a < n$ and L with Frag(v, a, L) and $L[n-a] = e$.

Definition 2.15 (Acknowledged)**.**

(i) Acked(v, n, q): $q =$ primary(v), or the history holds PREPAREOK(v, o, q) for some $o \geq n$.

(ii) QuorumAcked(v, n): there are Q distinct identities $q < N$ with Acked(v, n, q).

Definition 2.16 (Committed)**.** Committed(v, n, e): Holds(v, n, e) and QuorumAcked(v, n).

Definition 2.17 (Backed)**.** Backed(v, k): for every op-number $n \leq k$,

$$\exists\, e,\ v' \leq v. \quad \text{Committed}(v', n, e) \wedge \text{Holds}(v, n, e).$$

Lemma 2.18 (The history is monotone)**.** Vote*,* Frag*,* Holds*,* Acked*,* Committed*, and* Backed *are preserved by every step.*

Proof. Each is defined by the existence of elements of *sent* and *started*, which only grow. □

2.3 The invariant

Definition 2.19 (Replica-consistency)**.** At every replica: $k_r \le n_r$; $\ell_r \le v_r$; if $status_r$ is NORMAL or TRANSFER then $\ell_r = v_r$; if $catching_r$ then $status_r =$ VIEW-CHANGE; if $status_r =$ RECOVERING then $L_r = \varepsilon$ and $k_r = 0$.

Definition 2.20 (Message-consistency)**.** Every message in *sent* has fields that agree with one another: a StartView or DoViewChange log is as long as its op-number and its commit number does not exceed it; a NewState log is as long as $b - a$ with $a \le b$ and $k \le b$; a DoViewChange has $\ell \le v$; a Prepare has $n > 0$; a RecoveryResponse state has $k \le |L|$.

Definition 2.21 (View-agreement)**.**

(i) Within a view, an op-number determines its entry: $\text{Holds}(v, n, e) \wedge \text{Holds}(v, n, e') \Rightarrow e = e'$.

(ii) Every replica's log agrees with the fragments of the view it came from: $L_r[n] = e \wedge \text{Holds}(\ell_r, n, e') \Rightarrow e = e'$.

Remark 2.22. View-agreement says nothing across views: op-number n may name different entries in views 4 and 5, which is the paper's own observation (§4.2) that two operations can be assigned the same op-number only across a view change, and Commit-survival governs which of them matter. It is the property Raft calls log matching, with views for terms and message fragments alongside replica logs, and it is what lets the rest of the invariant say "the entry at op-number n in view v" and mean one thing.

Definition 2.23 (Commit-quorums)**.** $\text{Backed}(\ell_r, k_r)$ for every replica; and for every message in *sent* carrying a commit number k in view v, $\text{Backed}(v, k)$, a vote being judged by its last normal view.

Definition 2.24 (Commit-survival)**.** If $\text{Committed}(v', n, e)$ and $v' < v$, then every StartView log of v, every vote from v, every recovery state of v, every NewState segment of v that reaches n, every Prepare of v for op-number n, and every non-recovering replica with $\ell_r = v$ holds e at n.

Remark 2.25. This is the correctness condition of Liskov and Cowling [1, §8.1], that "every committed operation survives into all subsequent views in the same position in the serial order", stated over every fragment of a view rather than over the view as a whole.

Definition 2.26 (The invariant $\mathcal{I}$)**.** Replica-consistency, Message-consistency, View-agreement, Commit-quorums, and Commit-survival, together with the following clauses. Appendix B lists each by its name in the mechanised proof.

(a) *Views only rise.* No message in *sent*, and no vote, carries a view above its sender's current view. A replica in VIEW-CHANGE status has $\ell_r < v_r$; a vote for u is from a view $\ell < u$; the primary of a started view v has $\ell \ge v$.

(b) *Chosen from a quorum, once.* A view is started at most once. A started view's StartView log extends the best of the Q votes, from distinct replicas, it was started from. Every whole log of a started view, and every NewState of it, reaches at least as far as that best vote's log.

(c) *Votes cover acknowledgements.* A vote from x covers every op-number x acknowledged in the vote's view ℓ; if x is the primary of ℓ, it covers every fragment of ℓ.

(d) *Votes come after acknowledgements.* If x acknowledged in view v, or is the primary of a started view v, and cast a vote for $u > v$, that vote is from a view $\ell \ge v$.

(e) *Primary is longest.* While v is the last normal view of $\text{primary}(v)$ and it is not recovering, every fragment of v, every log of a replica last normal in v, and every acknowledgement in v is within its log.

(f) *Acknowledgements are kept.* If q sent $\text{PrepareOk}(v, o, q)$ then $\ell_q \ge v$, and while $\ell_q = v$ and q is not recovering, $n_q \ge o$. A normal primary's record A_r holds only acknowledgements of its own op-numbers in its own view, each acknowledger once.

Table 1: Notation. Subscript r names the replica; it is dropped when only one replica is in view.

Symbol	Meaning
N	number of replicas; identities are $0, \ldots, N-1$
f	$\lfloor N/2 \rfloor$, the failures tolerated
Q	$f+1$, the quorum size
primary(v)	$v \bmod N$, the primary of view v
v_r	view number of replica r
ℓ_r	last normal view: the latest view r was normal in
L_r	log of r, a sequence of entries with op-numbers $1, \ldots, \lvert L_r \rvert$
n_r	$\lvert L_r \rvert$, the op-number of the latest entry
k_r	commit number: the entries with op-numbers $1, \ldots, k_r$ are committed
σ_r	state of r's state machine
A_r, S_r, V_r, R_r, C_r	acknowledgements, view-change senders, votes, recovery responses, client table
$e = (c, s, op)$	a log entry: client, request number, operation
$L[n]$, $L(a..b]$, $L \cdot e$	entry with op-number n; op-numbers $a+1, \ldots, b$; append
(ℓ, L, k)	a vote: last normal view, log, commit number
Σ	a cluster state
sent	every (to, m) ever sent; never shrinks
started	every (v, V): view v was started from votes V
$\mathcal{R}$	the replicas whose status is not recovering
Sent(to, m)	$(to, m) \in$ *sent*
Vote(u, x, d)	x cast vote d for view u, sent or recorded
Frag(v, a, L)	the history holds L as view v's entries after op-number a
Holds(v, n, e)	some fragment of view v has e at op-number n
Acked(v, n, q)	replica q acknowledged op-number n in view v
QuorumAcked(v, n)	Q distinct replicas acknowledged n in v
Committed(v, n, e)	Holds(v, n, e) and QuorumAcked(v, n)
Backed(v, k)	every op-number up to k is committed with the entry v holds
$\mathcal{I}$	the invariant, Definition 2.26

(g) *Recovery covers acknowledgements.* A recovery state is the log of the primary of its view, answers a RECOVERY with its nonce, and, when it answers a replica now recovering, covers every op-number that replica acknowledged in the state's view.

(h) *Bookkeeping.* Replica i of the cluster has identity i; there are N replicas; every message names a sender below N; a replica catching up or in state transfer is not its view's primary; no PREPARE or COMMIT is addressed to its view's primary; every entry of every log is held by some fragment of the replica's last normal view; two replicas with the same last normal view agree; every view above zero with a fragment, an acknowledgement, or a non-recovering replica was started, and every started view has a STARTVIEW; what a replica has recorded in V_r or R_r was sent to it; between steps every outbox is empty; and no internal guard of INSTALLLOG or COMMITUPTO has failed.

Remark 2.27. The three safety corrections of Appendix A are each one clause. Truncation in state transfer would falsify (c). Diskless recovery would falsify (a), since a recovered replica's view could go down. A decreasing commit number would falsify COMMIT-QUORUMS at the replica that adopted it.

Table 1 collects the notation.

3 The framework

This section states what each handler guarantees and proves the theorem from those guarantees. The handlers themselves are in Sections 4 to 6, each of which ends by proving the interface lemmas stated here.

What is assumed and what is proved. The line is drawn where the protocol's control ends. Assumption 2.8 is about the world the protocol runs in: the network, what a crash keeps, when timers fire, where a nonce comes from. Nothing the protocol produces is assumed. A replica's fields are written by its handlers, and a message's fields by the replica that sends it, so any fact about them is the protocol's responsibility and must come out of the induction. That includes the two facts every handler relies on to run at all: that its own commit number is within its own log, and that the message it was handed has fields that agree with each other. These are the input requirements of the handlers, in the sense that a subroutine has preconditions its callers must meet. The callers here are the handler that ran before, for a replica's state, and the sender, for a message. We do not assume the callers comply. We prove it at every call site: Replica-consistency is proved once for every handler, and Message-consistency once for every send. Everything above them in the invariant is proved the same way, one handler at a time, from the interface lemmas below.

Where a commit is born. Committed(v, n, e) is a fact about the history: Q acknowledgements of n in v exist. It therefore comes into existence at the moment the Qth PrepareOk is *sent*, not when the primary receives it, and the obligation to show that every later view holds e (Commit-survival) falls on the sender. Three handlers send acknowledgements, Prepare, NewState, and StartView, and each of their interface lemmas carries that obligation. The primary's handler for PrepareOk only reads a quorum that already exists.

3.1 Interfaces of the handlers

Every interface lemma has the same shape. Precondition is what the handler may assume, always including $\mathcal{I}$. Effect is what it changes and sends. Guarantee is what holds afterwards: $\mathcal{I}$ again, plus the fact the step establishes that the rest of the proof uses. A handler whose guard fails has no effect and trivially preserves $\mathcal{I}$.

One guarantee is shared by every handler and needs no assumption about the message received, so it is stated once.

Lemma 3.1 (Local)**.** *Every handler of Sections 4 to 6, given any message, leaves the replica satisfying Replica-consistency (Definition 2.19): its commit number within its log, its last normal view not ahead of its view, and its status consistent with both. Every message it sends satisfies Message-consistency (Definition 2.20).*

Proof. k_r rises only in CommitUpTo, which stops at n_r; a log is replaced only in InstallLog, which refuses one shorter than k_r; ℓ_r is set only in EnterNormal, to v_r; v_r is set only to a larger value or, on recovery, to the persisted value. Every log sent is sent with its own length and the sender's commit number, which Replica-consistency bounds. □

The remaining lemmas are one per handler.

Lemma 3.2 (Request)**.** Precondition *$\mathcal{I}$; the replica is the normal primary of v_r and the request is new for its client.*

Effect *Appends e at op-number $n_r + 1$, records its own acknowledgement of it, and sends* Prepare$(v_r, n_r + 1, e, k_r)$ *to the others.*

Guarantee *$\mathcal{I}$. The new fragment* $\mathrm{Frag}(v_r, n_r, [e])$ *lies beyond every existing fragment of v_r.*

Lemma 3.3 (Prepare). PRECONDITION *$\mathcal{I}$; the message is accepted: $v = v_r$ and the replica is a normal backup.*

EFFECT *If $n = n_r + 1$, appends e; commits up to $\min(k, n_r)$; sends PREPAREOK$(v_r, n_r, \mathrm{id}_r)$ to the primary. If $n > n_r + 1$, enters state transfer instead.*

GUARANTEE *$\mathcal{I}$. In particular, if the acknowledgement completes a quorum for some op-number $m \le n_r$ in v_r, then* Committed$(v_r, m, L_r[m])$ *now holds and every started view above v_r holds $L_r[m]$ at m.*

Lemma 3.4 (PrepareOk). PRECONDITION *$\mathcal{I}$; the replica is the normal primary of v_r, $n > k_r$, and $n \in \operatorname{dom} A_r$.*

EFFECT *Adds i to $A_r[n]$; if that makes $|A_r[n]| = Q$, commits up to n and replies to the clients of the committed entries.*

GUARANTEE *$\mathcal{I}$. If the quorum completed, then* QuorumAcked(v_r, m) *already holds in the history for every $m \le n$, and hence* Backed(v_r, n).

Lemma 3.5 (Commit). PRECONDITION *$\mathcal{I}$; the message is accepted.*

EFFECT *Commits up to k, or enters state transfer if $k > n_r$.*

GUARANTEE *$\mathcal{I}$.*

Lemma 3.6 (GetState). PRECONDITION *$\mathcal{I}$; the replica is normal in $v = v_r$ and $n \le n_r$.*

EFFECT *Sends NEWSTATE$(v, L_r(n..n_r], n, n_r, k_r)$ to i.*

GUARANTEE *$\mathcal{I}$. The new fragment* Frag$(v, n, L_r(n..n_r])$ *agrees with every fragment of v.*

Lemma 3.7 (NewState). PRECONDITION *$\mathcal{I}$; $v = v_r$.*

EFFECT *In TRANSFER status, appends the part of L past n_r, commits up to k, becomes normal, and acknowledges its whole log. In VIEW-CHANGE status while catching up with $a = k_r$, replaces $L_r(k_r..n_r]$ by L, commits up to k, becomes normal in v_r, and acknowledges its whole log.*

GUARANTEE *$\mathcal{I}$. Afterwards $\ell_r = v_r$ and L_r agrees with every fragment of v_r. The acknowledgement carries the same guarantee as in Lemma 3.3.*

Lemma 3.8 (StartViewChange and DoViewChange). PRECONDITION *$\mathcal{I}$; $v \ge v_r$.*

EFFECT *If $v > v_r$, adopts v in VIEW-CHANGE status and announces it. Records the sender. On f senders, casts its vote (ℓ_r, L_r, k_r): sends it to* primary(v_r)*, or records it if it is the primary. On a DOVIEWCHANGE as primary, records the vote; on the Qth, starts the view (Lemma 3.9).*

GUARANTEE *$\mathcal{I}$. A vote cast by r satisfies clauses (a), (c), and (d) of Definition 2.26: it is from $\ell_r < v_r$, it covers every op-number r acknowledged in ℓ_r, and it comes after every acknowledgement r made.*

Lemma 3.9 (Starting a view). PRECONDITION *$\mathcal{I}$; the replica is* primary(v_r) *in VIEW-CHANGE status and holds Q votes V from distinct replicas, each sent to it for v_r or its own.*

EFFECT *Adds (v_r, V) to started; installs the log L of the best vote; commits up to the greatest k in V; becomes normal in v_r; sends STARTVIEW$(v_r, L, |L|, k_r)$ to the others.*

GUARANTEE *$\mathcal{I}$. For every $u < v_r$ and every* Committed(u, n, e)*: $L[n] = e$. The guards of INSTALLLOG and COMMITUPTO hold.*

Lemma 3.10 (StartView). PRECONDITION *$\mathcal{I}$; $v > v_r$, or $v = v_r$ in VIEW-CHANGE status.*

EFFECT *Adopts v; installs L; commits up to k; becomes normal in v; acknowledges its whole log.*

GUARANTEE *$\mathcal{I}$. Afterwards $\ell_r = v$ and $L_r = L$. The acknowledgement carries the same guarantee as in Lemma 3.3. The guard of INSTALLLOG holds.*

Lemma 3.11 (Recovery and RecoveryResponse). PRECONDITION *$\mathcal{I}$.*

EFFECT *On RECOVERY carrying $v > v_r$, adopts v. On RECOVERY in normal status, answers with v_r and, if primary, with (L_r, k_r). On the Qth RECOVERYRESPONSE for its nonce, if one is from the primary of the latest view v^* among them and $v^* \ge v_r$, adopts that primary's log and commit number and becomes normal in v^*.*

GUARANTEE $\mathcal{I}$. *A recovery state sent is a fragment of its view that covers every acknowledgement in that view, clause (g).*

Lemma 3.12 (Recover)**.** PRECONDITION $\mathcal{I}$*; the persisted view is* v_r*; the nonce is fresh (Assumption 2.8(iv)).*
EFFECT *Resets the replica to its initial state except* $v_r = \ell_r = v_r$*, in* RECOVERING *status with the fresh nonce, and sends* RECOVERY(id_r, x, v_r) *to the others.*
GUARANTEE $\mathcal{I}$. *In particular clause (a): no message the replica ever sent is ahead of* v_r*; and clause (g) for the new nonce, vacuously, since no response to it exists yet.*

Lemma 3.13 (Idle)**.** PRECONDITION $\mathcal{I}$.
EFFECT *Re-sends: the primary its* COMMIT *and every uncommitted* PREPARE*; a recovering replica its* RECOVERY*; a replica in transfer its* GETSTATE*; a replica in view change its* STARTVIEWCHANGE *and, if it has voted, its vote. A backup or a replica in view change whose timer fires starts view* $v_r + 1$.
GUARANTEE $\mathcal{I}$. *Every fragment re-sent is one the history already holds.*

3.2 The invariant is inductive

Lemma 3.14 (Initial)**.** $\mathcal{I}$ *holds in the initial cluster.*

Proof. Nothing has been sent, so every clause about the history is vacuous; every replica is fresh, so every clause about replica state holds. □

Theorem 3.15 ($\mathcal{I}$ is inductive)**.** $\mathcal{I}$ *holds in the initial cluster, and if* $\mathcal{I}(\Sigma)$ *and* $\Sigma \to \Sigma'$ *then* $\mathcal{I}(\Sigma')$*. Hence* $\mathcal{I}$ *holds in every reachable cluster.*

Proof. Lemma 3.14 for the initial cluster. A step runs one handler at one replica; every handler is covered by exactly one of Lemmas 3.2 to 3.13, whose guarantee includes $\mathcal{I}$, or its guard fails and nothing changes. Induction on the number of steps. □

3.3 Proof of Theorem 1.1

Lemma 3.16 (Committed entries are what the view holds)**.** *If* $\mathcal{I}(\Sigma)$ *and* $n \le k_r$*, there are* e *and* $v' \le \ell_r$ *with* $L_r[n] = e$*,* $\mathrm{Committed}(v', n, e)$*, and* $\mathrm{Holds}(\ell_r, n, e)$.

Proof. COMMIT-QUORUMS gives e, v', and $\mathrm{Holds}(\ell_r, n, e)$; VIEW-AGREEMENT identifies e with $L_r[n]$. □

Lemma 3.17 (Later views hold earlier commits)**.** *If* $\mathcal{I}(\Sigma)$*,* $\mathrm{Committed}(v', n, e)$*,* $v' < v$*, and* $\mathrm{Holds}(v, n, e')$*, then* $e' = e$.

Proof. Cases on the fragment that holds e', each a clause of COMMIT-SURVIVAL. □

Lemma 3.18 (An acknowledger holds the entry)**.** *If* $\mathcal{I}(\Sigma)$*,* $\mathrm{Committed}(v, n, e)$*, and* $z \in \mathcal{R}$ *with* $\mathrm{Acked}(v, n, \mathrm{id}_z)$*, then* $L_z[n] = e$.

Proof. First $\ell_z \ge v$: by clause (f) if z sent the acknowledgement, and by clause (a) if z is the primary of v, since v was started (clause (h)) and its primary has $\ell \ge v$. If $\ell_z > v$, the replica clause of COMMIT-SURVIVAL gives $L_z[n] = e$. If $\ell_z = v$, the log reaches n: by clause (f) for a sent acknowledgement, by clause (e) for the primary, whose log covers every fragment of v including the one holding e; and VIEW-AGREEMENT(ii) identifies $L_z[n]$ with e. □

Proof of Theorem 1.1. Let Σ be reachable; $\mathcal{I}(\Sigma)$ by Theorem 3.15.

Prefix agreement. For $n \le k_a, k_b$, Lemma 3.16 gives e_a committed in v_a and e_b committed in v_b; if $v_a < v_b$ then $e_b = e_a$ by Lemma 3.17, symmetrically if $v_b < v_a$, and by VIEW-AGREEMENT(i) if equal.

Durability. Let $r \in \mathcal{R}$ and $n \le k_r$. By Lemma 3.16, $e = L_r[n]$ is committed in some v, so some set X of Q distinct identities acknowledged n in v. Let P be the identities of $\mathcal{R}$. By Lemma 2.10, $|X \cap P| \ge Q + |\mathcal{R}| - N \ge |\mathcal{R}| + 1 - Q$, using $2Q \ge N + 1$. Every replica in $X \cap P$ holds e at n by Lemma 3.18.

Commit bounded. REPLICA-CONSISTENCY. □

4 Normal operation and state transfer

4.1 Overview

The primary orders requests by appending them to its log and telling the backups with PREPARE. A backup acknowledges with its whole log length, so one PREPAREOK covers every op-number up to it. The primary commits an op-number, and everything before it, on Q acknowledgements counting its own, and tells the backups through the commit number in the next PREPARE or an idle COMMIT. A backup that finds a gap asks for the missing log with GETSTATE; a replica that learns of a later view keeps its committed prefix and asks for the rest of the new view's log from there. Algorithm 1 gives the helpers every section uses; Algorithm 2 the handlers.

4.2 Analysis

Lemma 4.1 (Why views agree)**.** *Let $p = \text{primary}(v)$. Then:*

(i) One tenure. *p is normal in v during a single interval of the execution: once p leaves view v, by moving to a later view or by crashing, it never returns to v.*

(ii) Append only. *During that interval L_p only grows, and afterwards, until p recovers or moves on, it is frozen.*

(iii) Everything comes from it. *Every fragment of v in the history is a prefix of the log p held at the time the fragment was sent.*

Hence all fragments of v are prefixes of one sequence, and any two agree wherever they overlap, which is VIEW-AGREEMENT.

Proof. (i) is clause (a): a replica's view number never decreases, including across recovery, where it restarts at the persisted view. (ii): while normal, p changes its log only in APPEND on a REQUEST; INSTALLLOG runs only on entering a later view or on recovery, which is clause (e). (iii): a PREPARE or NEWSTATE of v is sent by p from its own log, or by a backup from a log that VIEW-AGREEMENT says agrees with p's; a STARTVIEW of v is p's log when it started v; a vote from v is the log of a replica last normal in v, which by VIEW-AGREEMENT agrees with p's. □

Lemma 4.2 (A new commit survives)**.** *Suppose $\mathcal{I}(\Sigma)$ and replica r, normal in v with a log that agrees with every fragment of v, sends $\textsc{PrepareOk}(v, n_r, r)$. If for some $m \le n_r$ this completes a quorum, so that* $\text{Committed}(v, m, L_r[m])$ *holds in Σ', then in Σ' every fragment of every started view $u > v$ that reaches m, and every non-recovering replica last normal in such a u, holds $L_r[m]$ at m.*

Proof. Strong induction on u over the started views above v. Let u be started from V; by the induction hypothesis every vote in V from a view strictly between v and u holds $L_r[m]$ at m, since such a vote is a fragment of that view. Lemma 5.1 then gives that the best vote of V holds $L_r[m]$ at m. By clause (b) the STARTVIEW of u extends the best vote, so it holds $L_r[m]$ at m; every other fragment of u and every replica last normal in u reaches m by clause (b) and agrees with the STARTVIEW by VIEW-AGREEMENT. □

Algorithm 1 Log and status helpers. r is the replica.

procedure APPEND(e)
  $L_r \leftarrow L_r \cdot e$; $C_r[e.c] \leftarrow (e.s, \bot)$
end procedure
procedure INSTALLLOG(L)
  require $|L| \geq k_r$ ▷ Lemma 3.1
  $L_r \leftarrow L$; rebuild C_r from L, keeping replies for op-numbers up to k_r
end procedure
procedure COMMITUPTO(k, *reply*)
  while $k_r < k$ **do** ▷ App. A.2
    require $k_r < n_r$ ▷ Lemma 3.1
    $k_r \leftarrow k_r + 1$
    $(\sigma_r, x) \leftarrow \text{apply}(\sigma_r, L_r[k_r].op)$; record x in $C_r[L_r[k_r].c]$
    if *reply* **then** reply $(v_r, L_r[k_r].c, L_r[k_r].s, x)$
    end if
  end while
end procedure
procedure ENTERNORMAL
  $status_r \leftarrow$ NORMAL; $\ell_r \leftarrow v_r$; $S_r, V_r \leftarrow \emptyset$; $voted_r, catching_r \leftarrow$ false
end procedure
procedure STATETRANSFER
  $status_r \leftarrow$ TRANSFER
  send GETSTATE(id_r, v_r, n_r) to the primary
end procedure
procedure CATCHUP(v)
  if $v = v_r \wedge catching_r$ **then return**
  end if
  $v_r \leftarrow v$; $status_r \leftarrow$ VIEW-CHANGE; $catching_r \leftarrow$ true; $S_r, V_r \leftarrow \emptyset$; $voted_r \leftarrow$ false
  send GETSTATE(id_r, v, k_r) to primary(v) ▷ App. A.4
end procedure
procedure ACCEPT(v) ▷ may this normal-case message be processed?
  if $v < v_r$ **then return** false
  end if
  if $v > v_r$ **then** CATCHUP(v); **return** false
  end if
  if $status_r =$ NORMAL **then return** $\neg$isPrimary(r)
  end if
  if $status_r =$ VIEW-CHANGE **then** CATCHUP(v)
  end if
  return false
end procedure

Algorithm 2 Normal operation and state transfer.

on REQUEST(c, s, op) ▷ Lemma 3.2
1: **require** isPrimary$(r) \wedge status_r =$ NORMAL
2: **if** $C_r[c]$ undefined or $s > C_r[c].s$ **then**
3: APPEND$((c, s, op))$; $A_r[n_r] \leftarrow \{\mathrm{id}_r\}$
4: send PREPARE$(v_r, n_r, (c, s, op), k_r)$ to others
5: **else if** $s = C_r[c].s$ and $C_r[c]$ holds a reply x **then**
6: reply (v_r, c, s, x)
7: **end if**
on PREPARE(v, n, e, k) ▷ Lemma 3.3
8: **require** ACCEPT(v)
9: **if** $n > n_r + 1$ **then** STATETRANSFER; **return**
10: **end if**
11: **if** $n = n_r + 1$ **then** APPEND(e)
12: **end if**
13: COMMITUPTO$(\min(k, n_r)$, false)
14: send PREPAREOK$(v_r, n_r, \mathrm{id}_r)$ to the primary ▷ Lemma 4.2
on PREPAREOK(v, n, i) ▷ Lemma 3.4
15: **require** $v = v_r \wedge$ isPrimary$(r) \wedge status_r =$ NORMAL $\wedge\, n > k_r \wedge n \in \operatorname{dom} A_r$
16: $A_r[n] \leftarrow A_r[n] \cup \{i\}$
17: **if** i was new and $|A_r[n]| = Q$ **then**
18: COMMITUPTO$(n$, true); remove every $n' \leq n$ from A_r
19: **end if**
on COMMIT(v, k) ▷ Lemma 3.5
20: **require** ACCEPT(v)
21: **if** $k > n_r$ **then** STATETRANSFER
22: **else** COMMITUPTO$(k$, false)
23: **end if**
on GETSTATE(i, v, n) ▷ Lemma 3.6
24: **require** $status_r =$ NORMAL $\wedge\, v = v_r \wedge n \leq n_r$
25: send NEWSTATE$(v, L_r(n..n_r], n, n_r, k_r)$ to i
on NEWSTATE(v, L, a, b, k) ▷ Lemma 3.7
26: **require** $v = v_r \wedge |L| = b - a$
27: **if** $status_r =$ TRANSFER **then**
28: **if** $a > n_r \vee b \leq n_r$ **then return**
29: **end if**
30: APPEND(e) for each e in $L(n_r - a..b - a]$, in order
31: COMMITUPTO$(k$, false); $status_r \leftarrow$ NORMAL
32: send PREPAREOK$(v_r, n_r, \mathrm{id}_r)$ to the primary ▷ Lemma 4.2
33: **else if** $status_r =$ VIEW-CHANGE $\wedge\, catching_r \wedge a = k_r$ **then**
34: INSTALLLOG$(L_r(0..a] \cdot L)$ ▷ App. A.4
35: COMMITUPTO$(k$, false); ENTERNORMAL
36: send PREPAREOK$(v_r, n_r, \mathrm{id}_r)$ to the primary ▷ Lemma 4.2
37: **end if**

Proof of Lemma 3.2. The new entry is at op-number $n_r + 1$, and by clause (e) every fragment of v_r ends at or before n_r, so VIEW-AGREEMENT holds for the new fragment. The PREPARE carries k_r, backed by COMMIT-QUORUMS. Clause (e) is maintained since the primary's log grew with the fragment. □

Proof of Lemma 3.3. The appended entry is the fragment $\text{Frag}(v, n-1, [e])$, which agrees with the replica's log by VIEW-AGREEMENT for $\ell_r = v_r = v$. The commit is bounded by the message's k, which is backed in v by COMMIT-QUORUMS, and by n_r, so $\text{Backed}(\ell_r, k_r)$. The acknowledgement names n_r, which is within the primary's log by clause (e) since every entry of L_r is a fragment of v; clause (f) holds for it since the replica is normal in v and holds its log. If the acknowledgement completes a quorum, Lemma 4.2 gives COMMIT-SURVIVAL for the new Committed facts. □

Proof of Lemma 3.4. By clause (f) every member of $A_r[n]$ other than the primary sent a PREPAREOK for op-number n in v_r, and the members are distinct, so $|A_r[n]| = Q$ gives $\text{QuorumAcked}(v_r, m)$ for all $m \le n$. The entries at those op-numbers are held by v_r (they are the primary's own fragments), so $\text{Committed}(v_r, m, L_r[m])$ and $\text{Backed}(v_r, n)$; COMMIT-QUORUMS follows for the new $k_r = n$, and the guard of COMMITUPTO holds since $n \le n_r$ by clause (f). □

Proof of Lemma 3.5. COMMIT-QUORUMS at the message gives $\text{Backed}(v, k)$, and $v = \ell_r$. □

Proof of Lemma 3.6. The sender is normal in v, so $\ell_r = v$ and by VIEW-AGREEMENT its log agrees with every fragment of v; the segment sent is a piece of that log. Its commit number is backed by COMMIT-QUORUMS at the sender, and its end n_r reaches the base of v by clause (b). □

Proof of Lemma 3.7. In TRANSFER status $\ell_r = v_r = v$ by REPLICA-CONSISTENCY, and the appended entries are a fragment of v agreeing with its log by VIEW-AGREEMENT. While catching up, the kept prefix $L_r(0..k_r]$ is backed in $\ell_r < v$ by COMMIT-QUORUMS, so by COMMIT-SURVIVAL the fragment of v holds the same entries at those op-numbers; and the fragment reaches k_r because by clause (b) it reaches the base of v, which by Lemma 5.1 holds every committed entry. The installed suffix is the fragment itself. Hence the new L_r agrees with every fragment of v, which is VIEW-AGREEMENT for the new $\ell_r = v$, and $\text{Backed}(v, k_r)$ by COMMIT-SURVIVAL and the message's k. The acknowledgement is as in Lemma 3.3. □

5 View change

5.1 Overview

A replica that stops hearing from the primary moves to the next view and says so. A replica that hears f others say so votes: it sends the new primary its log tagged with the view the log came from. The new primary takes the log from the vote with the latest such view, longest on a tie, which by Lemma 5.1 holds everything any earlier view committed, and announces it with STARTVIEW. Algorithm 3 gives the helpers, Algorithm 4 the handlers, and Algorithm 5 the idle step whose timer starts it all.

5.2 Analysis

Lemma 5.1 (The chosen log holds earlier commits)**.** *Let* $\mathcal{I}(\Sigma)$*. Suppose* $\text{Holds}(v, n, e)$ *and a set* X *of* Q *distinct identities below* N *with* $\text{Acked}(v, n, x)$ *for every* $x \in X$*. Let* $u > v$*, and let* V *be* Q *votes for* u *from distinct replicas, each satisfying clauses (c) and (d), such that every vote in* V *from a view* ℓ *with* $v < \ell < u$ *holds* e *at* n*. Then the best vote of* V*, by* $(\ell, |L|)$*, holds* e *at* n*.*

Algorithm 3 View-change helpers.

procedure StartViewChange(v)
  $v_r \leftarrow v$; $status_r \leftarrow$ view-change; $catching_r \leftarrow$ false; $S_r, V_r \leftarrow \emptyset$; $voted_r \leftarrow$ false
  send StartViewChange(v, id_r) to others
  MaybeVote
end procedure
procedure MaybeVote
  if $status_r =$ view-change $\wedge \neg catching_r \wedge \neg voted_r \wedge |S_r| \geq f$ **then**
    $voted_r \leftarrow$ true; Vote
  end if
end procedure
procedure Vote ▷ Lemma 3.8
  if primary$(v_r) = \mathrm{id}_r$ **then** RecordVote$(\mathrm{id}_r, (\ell_r, L_r, k_r))$
  else send DoViewChange$(v_r, \mathrm{id}_r, \ell_r, L_r, n_r, k_r)$ to the primary
  end if
end procedure
procedure RecordVote(i, *vote*) ▷ Lemma 3.9
  $V_r[i] \leftarrow vote$
  if $|V_r| < Q$ **then return**
  end if
  best $\leftarrow$ the vote in V_r with the greatest $(\ell, |L|)$ lexicographically, ties to the highest identity ▷ Lemma 5.1
  $k^* \leftarrow \max\{\, k : (\ell, L, k) \in V_r \,\}$
  start v_r from V_r ▷ adds (v_r, V_r) to *started*
  InstallLog(*best*.L); CommitUpTo(k^*, true); EnterNormal
  $A_r \leftarrow \{\, n \mapsto \{\mathrm{id}_r\} : k_r < n \leq n_r \,\}$
  send StartView(v_r, L_r, n_r, k_r) to others
end procedure

Algorithm 4 View change.

on StartViewChange(v, i) ▷ Lemma 3.8
if $v < v_r$ **then return**
end if
if $v > v_r$ **then** StartViewChange(v) ▷ App. A.3
end if
if $status_r =$ view-change **then** $S_r \leftarrow S_r \cup \{i\}$; MaybeVote
else if $status_r =$ normal $\wedge$ isPrimary(r) **then** send StartView(v_r, L_r, n_r, k_r) to i
end if
on DoViewChange(v, i, ℓ, L, n, k) ▷ Lemma 3.8
require $v \geq v_r \wedge \mathrm{primary}(v) = \mathrm{id}_r \wedge |L| = n$
if $v > v_r$ **then** StartViewChange(v)
end if
if $status_r =$ normal **then** send StartView(v_r, L_r, n_r, k_r) to i
else RecordVote$(i, (\ell, L, k))$
end if
on StartView(v, L, n, k) ▷ Lemma 3.10
require $|L| = n \wedge (v > v_r \vee (v = v_r \wedge status_r =$ view-change$))$ ▷ App. A.3
$v_r \leftarrow v$; InstallLog(L); CommitUpTo$(k,$ false$)$; EnterNormal; $A_r \leftarrow \emptyset$
send PrepareOk$(v_r, n_r, \mathrm{id}_r)$ to the primary ▷ Lemma 4.2

Algorithm 5 Idle. "The timer fires" is decided by the replica's idle counters, which the environment's schedule drives (Assumption 2.8).

on idle ▷ Lemma 3.13

if $status_r = \text{NORMAL} \wedge \text{isPrimary}(r)$ **then**
  send $\text{COMMIT}(v_r, k_r)$ to others
  send $\text{PREPARE}(v_r, n, L_r[n], k_r)$ to others for each $n = k_r + 1, \ldots, n_r$ ▷ App. A.2
else if $status_r = \text{NORMAL}$ **then**
  if the timer fires **then** $\text{STARTVIEWCHANGE}(v_r + 1)$
  end if
else if $status_r = \text{RECOVERING}$ **then**
  send $\text{RECOVERY}(\text{id}_r, x_r, v_r)$ to others
else if $status_r = \text{TRANSFER}$ **then**
  STATETRANSFER; **if** the timer fires **then** $\text{STARTVIEWCHANGE}(v_r + 1)$
else if the timer fires **then**
  $\text{STARTVIEWCHANGE}(v_r + 1)$
else if $catching_r$ **then**
  send $\text{GETSTATE}(\text{id}_r, v_r, k_r)$ to the primary
else
  send $\text{STARTVIEWCHANGE}(v_r, \text{id}_r)$ to others; **if** $voted_r$ **then** VOTE
end if

Proof. By Lemma 2.10 some $x \in X$ cast a vote $d = (\ell, L, k)$ in V. By clause (d), $\ell \geq v$: x either sent a PREPAREOK in v before voting, or is the primary of v, which was started since it holds a fragment. If $\ell = v$: by clause (c) the vote covers n, as an op-number x acknowledged in v or, for the primary, as a fragment of v; and VIEW-AGREEMENT(i) gives $L[n] = e$. If $\ell > v$: the hypothesis gives $L[n] = e$, since $\ell < u$ by clause (a). So d holds e at n. Let $b = (\ell_b, L_b, k_b)$ be the best vote, so $(\ell_b, |L_b|) \geq (\ell, |L|)$. If $\ell_b = \ell$ then $|L_b| \geq |L| \geq n$ and $L_b[n] = e$ by VIEW-AGREEMENT(i), both being fragments of ℓ. If $\ell_b > \ell \geq v$ then b is a vote from a view strictly between v and u, and the hypothesis gives $L_b[n] = e$. □

Proof of Lemma 3.8. Adopting a larger view preserves clause (a). A vote is (ℓ_r, L_r, k_r) with ℓ_r the view L_r came from, so it is $\text{Frag}(\ell_r, 0, L_r)$, agreeing with ℓ_r's other fragments by VIEW-AGREEMENT; its k_r is backed in ℓ_r by COMMIT-QUORUMS; it is from $\ell_r < v_r$ by clause (a); by clause (f) it covers every op-number r acknowledged in ℓ_r, which is clause (c), and if r is the primary of ℓ_r it covers every fragment of ℓ_r by clause (e); and every acknowledgement r made in a view $v' < v_r$ was made while $\ell_r \geq v'$, since ℓ_r only rises, which is clause (d). A STARTVIEW sent by a normal primary in reply is its current log, a fragment of v_r by clause (e). □

Proof of Lemma 3.9. The Q votes are from distinct replicas and v_r was not started before, as its primary was not normal in it, so clause (b) holds for the new (v_r, V). For $u < v_r$ and $\text{Committed}(u, n, e)$, every vote in V from a view between u and v_r holds e at n by COMMIT-SURVIVAL, so Lemma 5.1 gives $best.L[n] = e$, which is COMMIT-SURVIVAL for the new STARTVIEW and for the primary itself. The installed log is the new view's only fragment, so VIEW-AGREEMENT. Each k in V is backed in its vote's $\ell < v_r$ by COMMIT-QUORUMS, and by COMMIT-SURVIVAL the chosen log holds the same entries, so $\text{Backed}(v_r, k^*)$; in particular $k^* \leq |best.L|$ and $k_r \leq |best.L|$, so both guards hold. □

Proof of Lemma 3.10. The installed log is the fragment of v that the STARTVIEW carries, which gives VIEW-AGREEMENT for $\ell_r = v$. The message's k is backed in v by COMMIT-QUORUMS, and the replica's previous k_r, backed in the old $\ell_r < v$, is held by v at the same op-numbers by COMMIT-SURVIVAL, so the new $k_r = \max$ is backed and the guard of INSTALLLOG holds. The guard refuses a STARTVIEW for the current view in normal status, which is what clause (f)

Algorithm 6 Recovery.

on RECOVERY(i, x, v) ▷ Lemma 3.11
1: **if** $v > v_r \wedge status_r \neq$ RECOVERING **then** STARTVIEWCHANGE(v); **return**
2: **end if**
3: **require** $status_r =$ NORMAL
4: $st \leftarrow (L_r, k_r)$ if isPrimary(r) else $\perp$
5: send RECOVERYRESPONSE$(v_r, x, \mathrm{id}_r, st)$ to i
on RECOVERYRESPONSE(v, x, i, st) ▷ Lemma 3.11
6: **require** $status_r =$ RECOVERING $\wedge\, x = x_r$
7: $R_r[i] \leftarrow (v, st)$
8: **if** $|R_r| < Q$ **then return**
9: **end if**
10: $v^* \leftarrow \max\{\, v' : (v', \_) \in \operatorname{ran} R_r \,\}$
11: **require** $v^* \geq v_r \wedge R_r[\mathrm{primary}(v^*)] = (v^*, (L, k))$ for some L, k
12: $R_r \leftarrow \emptyset$; $v_r \leftarrow v^*$; INSTALLLOG(L); COMMITUPTO$(k$, false$)$; ENTERNORMAL
on recover with persisted view v and fresh nonce x ▷ Lemma 3.12; App. A.5
13: reset to the initial state, then $status_r \leftarrow$ RECOVERING; $v_r, \ell_r \leftarrow v$; $x_r \leftarrow x$
14: send RECOVERY(id_r, x, v) to others

needs. The acknowledgement names $|L|$, within the primary's log by clause (b), and is as in Lemma 3.3. □

Proof of Lemma 3.13. The primary's re-sent PREPARE for op-number n carries $L_r[n]$, a fragment of v_r that the history already holds by clause (e) and VIEW-AGREEMENT; its COMMIT carries k_r, backed by COMMIT-QUORUMS. A re-sent vote is the case of Lemma 3.8. Starting view $v_r + 1$ is the case of Lemma 3.8 with $v > v_r$. □

6 Recovery

6.1 Overview

A replica that comes back with nothing but its persisted view number asks everyone what it missed. It waits for Q answers to its nonce, one of them from the primary of the latest view among them, and adopts that primary's log. Until then it does nothing else. Its RECOVERY carries the persisted view, so replicas behind it move forward first. Algorithm 6 gives the handlers.

6.2 Analysis

Proof of Lemma 3.11. A recovery state is the normal primary's log in v_r, a fragment of v_r by clause (e), which also gives that it covers every acknowledgement in v_r; it answers a RECOVERY with the nonce it carries; so clause (g). Installing it at the recovering replica makes L_r that fragment, so VIEW-AGREEMENT for $\ell_r = v^*$, and k is backed by COMMIT-QUORUMS at the message; the guard of INSTALLLOG holds since the recovering replica has $k_r = 0$. The recovering replica held nothing before, so nothing is lost, and $v^* \geq v_r$ keeps clause (a). A RECOVERY carrying $v > v_r$ moves the receiver to v, which is Lemma 3.8. □

Proof of Lemma 3.12. By Assumption 2.8(iii) the persisted view is the replica's v_r in Σ, and by clause (a) in Σ no message it sent is ahead of that, so clause (a) holds after the reset. REPLICA-CONSISTENCY holds for a recovering replica with an empty log. By Assumption 2.8(iv) no RECOVERYRESPONSE answers the new nonce, so clause (g) is vacuous for it. Every other clause about the replica is either vacuous for RECOVERING status or unchanged. □

7 Conclusion

Viewstamped Replication as published in 2012 is not safe as written, in two places that lose committed data and one that repeats an operation. This paper gives the corrected protocol as pseudocode and proves it safe through an inductive invariant over the history of every message ever sent, checked in Lean 4. The invariant is the contribution: it is the precise form of the paper's informal correctness argument, and each correction to the paper corresponds to a clause of it that the published version of the protocol would make false. What the proof does not cover is what clients see, which needs a model of clients and their retransmissions, and liveness, which needs a fairness assumption and a different kind of argument.

AI disclosure

This draft was prepared with Claude Code, an AI coding agent, under the author's direction. The agent read the cited papers and the mechanised development and drafted the text, the pseudocode of Sections 4 to 6, and the proof sketches; the agent also took part in writing the Lean development. What has been checked: the Lean proof, by the Lean kernel, with the axioms stated in Appendix B. What has not: no human has yet checked the Lean development against the paper's definitions and pseudocode, or the hand proofs against the Lean, line by line. The correspondence in Appendix B is the agent's, and a simulated peer review has already found one place where the pseudocode and the model differ. The author takes responsibility for the whole.

References

[1] Barbara Liskov and James Cowling. Viewstamped replication revisited. Technical Report MIT-CSAIL-TR-2012-021, MIT Computer Science and Artificial Intelligence Laboratory, July 2012. URL `https://dspace.mit.edu/entities/publication/80846d94-fcd3-40e6-87fb-8d91fe99a5d1`.

[2] Ellis Michael, Dan R. K. Ports, Naveen Kr. Sharma, and Adriana Szekeres. Recovering shared objects without stable storage. Technical report, University of Washington. Extended version of the paper in the 31st International Symposium on Distributed Computing (DISC), 2017. URL `https://drkp.net/papers/recovery-tr17.pdf`.

[3] Brian M. Oki and Barbara H. Liskov. Viewstamped replication: A new primary copy method to support highly-available distributed systems. In *Proceedings of the Seventh Annual ACM Symposium on Principles of Distributed Computing (PODC)*, pages 8–17, 1988.

[4] Jack Vanlightly. VR revisited: An analysis with TLA+. Blog series in six parts, 2022. URL `https://jack-vanlightly.com/analyses/2022/12/20/vr-revisited-an-analysis-with-tlaplus`. Parts 1–6, December 2022 to January 2023.

A Viewstamped Replication

This appendix presents the protocol of Liskov and Cowling [1] as published, and, where Sections 4 to 6 correct it, says why. Section numbers in parentheses refer to that paper.

A.1 Replicas, views, and quorums

A cluster has $N = 2f + 1$ replicas with identities $0, \ldots, N - 1$ and tolerates f crashes; a quorum is $Q = f + 1$ (§2.2). The cluster moves through numbered *views*. The primary of view v is replica primary$(v) = v \bmod N$; there is no election, only a change of view.

Every replica holds a *view number* v_r, a *status* that is normal, view-change, state-transfer, or recovering, a *log* L_r of operations, a *commit number* k_r giving the length of the committed prefix of the log, and the *last normal view* ℓ_r, the most recent view in which its status was normal (Figure 2 of the paper). Entries are numbered by *op-number* from 1. A *client table* records, per client, the number of its latest request and, once executed, the reply, so that a retransmitted request is answered from the table rather than run again.

The paper's replica also has a disk it never writes to. We add one write: v_r is persisted after every step, before anything the step produced is sent, and survives a crash. The reason is in Section A.5.

A.2 Normal operation

A client sends Request to the primary (§4.1). The primary appends the operation to its log, assigns it the next op number n, records its own acknowledgement, and sends Prepare(v, n, e, k) to the backups, carrying the entry e and the primary's commit number k. A backup accepts Prepare only in view v_r, only in normal status, and only in op-number order; it appends, commits up to $\min(k, |L_r|)$, and replies PrepareOk$(v, |L_r|, r)$, which acknowledges every op up to its log length. When the primary has Q acknowledgements for op n, counting its own, it commits every op up to n in order, applying each to the state machine and replying to its client. When idle it sends Commit(v, k) so that backups learn of commits without a new request.

Erratum: a backup that adopts a lower commit number executes operations twice

The paper's backups set their commit number from whatever message arrives. But a StartView can legitimately carry a commit number lower than the receiver's, since the new primary's log is chosen by last normal view and length and its commit number is the maximum among the DoViewChange messages it chose from, which may be behind a backup that committed more in the old view. A backup that adopts the lower number re-executes the operations between on the next Prepare. We make every commit $k_r := \max(k_r, k)$: the commit number never decreases, from any message.

Erratum: a lost message is never re-sent, so the cluster stalls

The paper ignores re-sending. The primary re-sends every uncommitted Prepare on each idle period along with Commit, and a replica waiting for NewState or for a view change re-sends its request on each idle period. Without this a single lost message stalls the cluster.

A.3 View change

A backup that does not hear from the primary within a timeout advances to view $v + 1$, sets its status to view-change, and sends StartViewChange$(v + 1, r)$ to every other replica (§4.2). A replica that receives StartViewChange for a view above its own moves to that view and does the same. On f such messages from other replicas for its current view, a replica sends DoViewChange$(v, r, \ell_r, L_r, |L_r|, k_r)$ to primary(v), carrying its log and the view that log is from. On Q of them the new primary takes the log of the one with the greatest $(\ell, |L|)$ in lexicographic order, sets its commit number to the greatest k among them, commits up to it, becomes normal in v with $\ell = v$, and sends StartView$(v, L, |L|, k)$ to the others. A backup that receives StartView installs the log, commits up to k subject to the rule above, becomes normal in v, and acknowledges its whole log with a PrepareOk.

The paper's argument (§8.1) is that a committed op is in Q logs, a view change reads Q logs, and the two quorums intersect; and that a replica stops accepting Prepare from the old view the moment it enters view-change status, so the old primary cannot commit anything the new primary does not see.

Erratum: read as an increment, "advances its view-number" lets a primary count votes cast for different views together

§4.2 says a replica that receives StartViewChange for a later view "advances its view-number", which can be read as adopting the received view or as incrementing by one. Under the increment reading a primary can hold DoViewChange messages for views v, $v+1$, and $v+2$ at once and count them together, which builds a log from messages that were not all for the same view. We adopt the received view number and increment only on a timeout.

Erratum: a StartView for the current view would discard entries the replica has acknowledged

A replica in normal status in view v must not accept a StartView for v. Such a message is one it has already acted on, or a duplicate, and installing its log would discard entries the replica has since accepted and acknowledged. We accept a StartView for the current view only in view-change status, and for a later view always.

Erratum: a view-change backoff reset on entering the view lets the previous primary start the next view change

A view change that fails, because the new primary is also down, is followed by another with a longer timeout. We double the timeout per failed attempt. The backoff must not be reset on entering normal status: if it is, the previous primary, freshly normal, times out first in the next view and starts a change that the others, still backing off, then join, round the ring without end. It is forgotten only after the new view has been stable for a full primary timeout.

A.4 State transfer

A replica that receives a Prepare past the end of its log, or a Commit past its log, has missed messages. It sends GetState$(r, v, |L_r|)$ to the primary and receives NewState(v, L, a, b, k) with the entries with op-numbers $a+1$ to b, which it appends (§5.2). A replica that learns of a later view than its own from a normal-case message is in a different situation: the uncommitted suffix of its log belongs to a view that has ended and may have been superseded. The paper says it should "set its op-number to its commit-number and remove all entries after this from its log" before asking for state.

Erratum: truncating the log before state transfer discards an entry the replica has acknowledged

Suppose backup r has acknowledged op n in view v, so that op n may be committed on the strength of r's acknowledgement. Before r learns of the commit it hears of view $v+1$, truncates to its commit number, and loses op n. The change to $v+1$ then fails and a change to $v+2$ follows, in which r sends a DoViewChange with a log that no longer has op n. The quorum that acknowledged op n and the quorum of DoViewChange senders in $v+2$ may intersect only in r, and then op n, which was committed, is gone. The argument of §8.1 needs every replica's DoViewChange to cover every op it has acknowledged, and truncation breaks exactly that. We have a replica catching up with a later view keep its log, enter view-change status marked as catching up, ask for the log after its *commit number*, and on NewState replace only the uncommitted suffix. If the new view's log has op n, r keeps it; if not, the new view was chosen from a quorum that did not need r's acknowledgement, and op n was never committed.

A.5 Recovery

A replica that restarts with no memory sets its status to recovering and sends Recovery(r, x, v) with a fresh nonce x (§4.3). Normal replicas answer RecoveryResponse with their view number, and the primary of that view includes its log and commit number. On Q responses for its nonce, one of them from the primary of the latest view among them, the replica adopts that primary's state, becomes normal, and rejoins. While recovering it takes part in nothing else.

The paper argues (§8.2) that this is safe without a disk because of the StartViewChange round: a replica that has sent DoViewChange did so only after f others announced the view, so if it crashes and recovers, those others are still in the new view or beyond and it will recover into it. The round is offered as the equivalent of the write of the view id to stable storage that the original protocol made at the end of every view change [3, §4].

Erratum: diskless recovery lets a replica forget that it started a view change

The round moves the problem one message earlier. A replica can forget it sent StartViewChange as easily as it can forget a DoViewChange. With three replicas: the replica that will be the new primary sends StartViewChange, crashes before anyone receives it, and recovers in the old view, because both other replicas are still there and answer with it. The delayed StartViewChange then arrives at the third replica, which advances its view and sends its own StartViewChange and, now having f of them, a DoViewChange back. The recovered replica, which has no memory of wanting the new view, receives both, counts them, and completes the view change. The old primary knows nothing of this and keeps committing. We persist v_r after every step; a recovered replica starts in that view, and its Recovery message carries it so that the others move forward on receipt. This is what the 1988 protocol did with its view id, and what the 2012 paper gave up to need no disk; it turns out to be the one write that cannot be given up.

A.6 What is left out

Reconfiguration (§7) is out of scope; a lost machine is replaced by one that recovers into the same replica identity. Client recovery (§4.5) is the client's problem. The optimisations of §5 and §6, checkpoints, witnesses, batching, and reads at the primary, are not treated; the recovery protocol sends the whole log.

B The mechanised proof

The proof of Theorem 1.1 is checked in Lean 4. This appendix states the mechanised theorem, its assumptions, and how the paper's definitions and lemmas map onto it. Names in `monospace` are Lean identifiers.

B.1 The theorem

```
theorem safety (m : Machine Op Output St) (sm : St) (config : Config)
    (htwo : 2 <= config.replicaCount)
    (s : System Op Output St) (h : Reachable m sm config s) :
    NoPanic s /\ PrefixAgreement s /\ Durability s
```

together with `commitBounded_of_reachable` and `sentWF_of_reachable`, which give commit boundedness and Message-consistency for every reachable cluster. The three depend on the axioms `propext`, `Quot.sound`, and `Classical.choice` only, with no `sorry`. `NoPanic` is the last item of clause (h): no internal guard of InstallLog or CommitUpTo has failed.

`Reachable` is Definition 2.7: the closure of the initial cluster under the four steps of Definition 2.6, where a *recover* step requires `NonceFresh`, that no RECOVERY in `sent` carries the nonce. This and `2 <= replicaCount` are the only hypotheses; they are Assumption 2.8(i) and (iv). Assumption 2.8(iii) is built into the *recover* step, which restarts the replica with the view number it held. Assumption 2.8(ii) is built into the model of time: the handlers keep the protocol's idle counters, and the environment chooses which replica idles when, so every schedule is a reachable execution.

The model's logs are indexed from zero. Op-number n of this paper is list index $n - 1$ there, and a commit number k means indices 0 to $k - 1$; the definitions below are stated in those terms.

B.2 The model and its correspondence to the code

The model is a twin of an implementation: one Lean definition per function of the implementation, with the same control flow, sending to an outbox and appending replies to a list. Where the implementation checks a guard inside INSTALLLOG or COMMITUPTO, the model sets a flag instead of stopping, so every handler is total and "no guard fails" is a property to prove, `NoPanic`. The model is held to the implementation by differential testing rather than by translation: a generator produces seeded traces of cluster steps, both sides replay each trace and print every replica's status, view, commit number, log, and applied operations, and every message and reply sent, after every step, and the test fails on the first differing line. Every executable twin of an invariant clause is evaluated on the same traces, which is how a candidate clause was tried before it was proved.

What the mechanised proof does not cover is what the paper does not claim: what clients see, since the model has no clients and requests arrive from the environment, and liveness.

B.3 Correspondence

This paper	Lean
Definitions 2.1 to 2.4	`Config`, `Replica`, `Message`
Definition 2.5, 2.6	`System`, `Step`, `System.step`
Lemma 2.10	`Config.quorum_intersect`, `filter_mem_length`
Definition 2.11	`DvcOf`
Definition 2.12, Holds	`Frag`, `Holds`
Acknowledged, Committed, Backed	`Acked`, `QuorumAcked`, `Committed`, `Backed`, `MsgBacked`
Replica-consistency	`LocalInv`, lifted as `AllLocal`
Message-consistency	`WF`, lifted as `SentWF`
View-agreement	`OneLogPerView`
Commit-quorums	`CommitsBacked`
Commit-survival	`Survives`
$\mathcal{I}$, Definition 2.26	`Inv`
Lemma 2.18	`Frag.mono`, `Holds.mono`, `Committed.mono`, `Backed.mono`
Lemma 3.1	`LocalInv.onMessage`, `.onIdle`, `.recover`; `OutboxWF.*`
Lemma 3.2	`Inv.onRequest`
Lemma 3.3	`Inv.onPrepare`
Lemma 3.4	`Inv.onPrepareOk`
Lemma 3.5	`Inv.onCommit`
Lemma 3.6	`Inv.onGetState`
Lemma 3.7	`Inv.onNewState`
Lemma 3.8	`Inv.onStartViewChange`, `.onDoViewChange`, `.startViewChange`, `.sendDoViewChange`
Lemma 3.9	`Inv.recordDoViewChange`
Lemma 3.10	`Inv.onStartView`
Lemma 3.11	`Inv.onRecovery`, `Inv.onRecoveryResponse`
Lemma 3.12	`Inv.recover`
Lemma 3.13	`Inv.onIdle`
Lemma 3.14	`Inv.init`
Theorem 3.15	`Inv.step`, `inv_of_reachable`
Lemma 3.16	`Inv.committed_entry`
Lemma 3.17	`Inv.survives_holds`
Lemma 3.18	`Inv.holds_of_acked`
Lemma 5.1	`Inv.bestHolds`
Lemma 4.2	`Inv.viewHolds_of_newAck`
Theorem 1.1(i)	`Inv.prefixAgreement`
Theorem 1.1(ii)	`Inv.durability`
Theorem 1.1(iii)	`commitBounded_of_reachable`

Lemma 4.1, why views agree, has no single counterpart: it is the explanation of View-agreement, whose preservation is one obligation of each handler lemma above. The clauses of Definition 2.26 are the following fields of `Inv`.

Clause	Lean fields
(a) Views only rise	`MessagesBelowView`, `DvcBelowView`, `ViewChangeBehind`, `DvcBehind`, `PrimaryStartedNormal`
(b) Chosen from a quorum, once	`StartViewChosen`, `StartedOnce`, `ViewExtendsBase`
(c) Votes cover acknowledgements	`DvcCoversAcks`, `DvcPrimaryCovers`
(d) Votes come after acknowledgements	`DvcAfterAcks`, `DvcAfterOwnView`
(e) Primary is longest	`PrimaryLongest`
(f) Acknowledgements are kept	`AcksHold`, `AcksCurrent`, `AcksStarted`
(g) Recovery covers acknowledgements	`RecoveryCoversAcks`, `RecoveryResponseNonce`, `RecoveryStateFromPrimary`, `RecoveryResponseNotSelf`, `RecordedRRs`
(h) Bookkeeping	`Ids`, `ReplicaCount`, `SenderIds`, `StartedIds`, `CatchingUpNotPrimary`, `TransferNotPrimary`, `PrimaryToOthers`, `Covered`, `ReplicasAgree`, `StartedViews`, `StartViewChangePos`, `RecordedDvcs`, `Drained`, `Clean`, `TwoReplicas`, `NoPanic`

B.4 How preservation is organised

A step at one replica produces a new replica, a list of messages, and at most one started view; `System.after` is the cluster with those applied. `StepOK` collects one obligation per field of `Inv`, each about the new replica, the new messages, and the new started view only; facts about old messages and untouched replicas transfer by Lemma 2.18, and `Inv.after` turns `StepOK` into `Inv` of the new cluster. A handler is a chain of helpers, and its proof follows the chain through the cluster as it will be once the replica's outbox is handed over (`System.drainOf`), one reusable link per kind of change: replacing a replica while keeping its log, installing a log, appending an entry, committing up to a backed bound, sending each kind of message.

The Survival obligation of a new `Committed` fact is discharged where the paragraph "Where a commit is born" in Section 3 says: in `Inv.onPrepare`, `Inv.onNewState`, and `Inv.onStartView`. The lemma used there, `Inv.viewHolds_of_newAck`, is a strong induction on the started view, and each step of it is `Inv.bestHolds`. The same lemma discharges the guards of InstallLog and CommitUpTo when a view is started (`Inv.recordDoViewChange`), when a StartView is adopted (`Inv.onStartView`), and when a catching-up replica takes a NewState (`Inv.onNewState`).

Every field of `Inv` also has an executable twin that is evaluated on the states of randomly generated executions of the model, and every clause was tried that way before it was proved. Several of the clauses in (a) to (h) were found by that check failing, or by a preservation proof not closing, rather than written down in advance; that is the usual way an inductive invariant is arrived at, and the reason the paper presents $\mathcal{I}$ as the contribution.